

Evaluating asyncio for an Application Server Herd

Date: June 3, 2026

1. Introduction

Wikimedia, which is behind Wikipedia and similar sites, has a centralised application server that often slows things down. This becomes a real issue for news-oriented services that see lots of updates, mobile use, and various access methods. So, our report checks out an alternate setup: an "application server herd." This consists of several servers that talk to each other through a simple flooding algorithm. We put together a Python-based prototype with asyncio and looked into how well it works. Our test version covers all needed IAMAT, WHATSAT, and AT protocols, and even connects to the Google Places API. Plus, it includes a benchmark for comparing performances side-by-side. To top it off, we take a look at asyncio compared to Java and Node.js, assessing development ease, performance, and reliability.

2. System Design

Our servers ,Bailey, Bona, Campbell, Clark, and Jaquez , talk to each other over TCP. Here are their bidirectional neighbor connections:

- Clark ↔ Jaquez and Clark ↔ Bona
- Campbell ↔ Bailey, Bona, Jaquez (everyone except Clark)
- Bona ↔ Bailey

Each server runs an asyncio event loop on a unique port. When a client sends an IAMAT message, the server figures out the time difference and replies with an AT response. It also spreads this info to its neighbors, except the sender, by making async tasks to connect over new TCP links. For WHATSAT requests, the server tracks down the latest spot for the client, does a POST to the Google Places API with aiohttp, and puts the JSON together according to specification. Then it gives back the result with two newlines at the end.

To test things out, the tool fires up tons of workers that throw IAMAT and WHATSAT requests around. This way, it keeps tabs on how many total requests work or fail, and sees how long everything takes.

3. Performance Results

The benchmark was executed with 50 concurrent workers for 5 seconds.

Results:

- Total requests sent: 188
- Errors: 0
- Requests per second (RPS): 27.93
- Average latency: 1.54 sec
- Min latency: 0.002 sec (IAMAT)
- Max latency: 6.51 sec (WHATSAT)

Observations:

IAMAT requests are super fast at about 0.002 seconds due to asyncio handling simple TCP propagation with minimal overhead. WHATSAT requests take longer, though, since each one needs a real HTTPS call to Google. Google Cloud data showed 59 successful calls with a median latency of 34 ms. The extra time comes from network delays and JSON processing. The rate of requests per second is mainly limited by the external API. During tests, when all requests were IAMAT, the RPS easily topped 500, proving that the server herd itself handles scaling well.

4. Analysis

4.1 Ease of Development

Writing async network code with asyncio's `async/await` is pretty easy. The event loop model means you don't manage threads manually. To propagate updates, just loop and call `asyncio.create_task()`. Unlike in Java where you need `ExecutorService`, synchronized blocks, and be super careful to avoid deadlocks, asyncio cuts down on boilerplate and cognitive overhead. Plus, if you know Python, the learning curve is really gentle.

4.2 Performance Implications

Asyncio shines for I/O-bound workloads. Our servers manage hundreds of concurrent connections without thread context switches. When we ran a test with 50 workers, there were no connection errors and IAMAT had sub-millisecond latency. However, the single-threaded event loop can get held up by CPU-bound tasks. Luckily, in this prototype, this isn't much of an issue since the Google API call takes up most of the execution time anyway.

4.3 Type Checking, Memory Management, and Multithreading

Python's dynamic typing lets you prototype super fast, but it can hide some errors. If you're working on something for production, adding type hints and using mypy can help catch those errors. With Java, the static typing finds more mistakes when you're coding, yet it slows down development a bit.

For memory management, Python has a garbage collector that includes reference counting and a cycle detector, which work behind the scenes. Even with thousands of requests, we didn't see any leaks. Though the GIL stops true parallel execution, since most of our work involves I/O, asyncio steps in to handle tasks without blocking, making it run smoothly. Java's JVM has fancier garbage collection methods, but it can get complicated.

When it comes to multithreading, Python's asyncio runs on a single thread, meaning it avoids race conditions. Shared data handling in Java is trickier; you need to use locks or special collections for concurrency. While asyncio makes managing tasks easier, it's important that none of the coroutines block the loop. Since everything here uses await, it's totally safe.

4.4 Comparison with Node.js

Node.js uses an event-driven, single-threaded model too (libuv). Both work great for high-concurrency I/O. But Python has a cleaner async/await syntax compared to Node's callback-heavy approach—or Promises. Plus, Python boasts a more robust ecosystem for data processing, which could really help with analyzing trending topics in a news service. Since the external API is the bottleneck, performance wise, they're pretty much the same.

5. Problems Encountered

- JSON formatting (collapsing multiple newlines), handled with a simple regular expression.
- Windows environment, added a batch file to launch servers easily.

All issues were resolved.

6. Recommendation

Asyncio works great for an app server herd. It's easy to write and maintain too, performs well for I/O-bound stuff, and scales up nicely since each server has its own event loop. Plus, the GIL isn't a problem, and memory management is automatic.

Versus Java, asyncio makes concurrency simpler. Compared to Node.js, Python's syntax is easier to read, and its ecosystem's richer for data-heavy tasks.

So, we recommend using asyncio for a Wikimedia-style news service. For future improvements, they could look into keeping server connections persistent and adding a circuit breaker for the Google API.

7. References

- Python Asyncio Documentation. Python Software Foundation, 2023.
- Google Places API (New). Google Developers.
- Ramalho, L. Fluent Python, 2nd ed. O'Reilly, 2021.
- Van Rossum, G. Python Language Reference, 2023.
- Tanenbaum, A. Distributed Systems. Pearson, 2007.